

GPU-Aware Design, Implementation, and Evaluation of Non-blocking Collective Benchmarks *

A. A. Awan
Dept of Computer Science
and Engineering
The Ohio State University
awan.10@osu.edu

K. Hamidouche
Dept of Computer Science
and Engineering
The Ohio State University
hamidouche.2@osu.edu

A. Venkatesh
Dept of Computer Science
and Engineering
The Ohio State University
venkatesh.19@osu.edu

J. Perkins
Dept of Computer Science
and Engineering
The Ohio State University
perkins.173@osu.edu

H. Subramoni
Dept of Computer Science
and Engineering
The Ohio State University
subramoni.1@osu.edu

D. K. Panda
Dept of Computer Science
and Engineering
The Ohio State University
panda.2@osu.edu

ABSTRACT

As we move towards efficient exascale systems, heterogeneous accelerators like NVIDIA GPUs are becoming a significant compute component of modern HPC clusters. It has become important to utilize every single cycle of every compute device available in the system. From NICs to GPUs to Co-processors, heterogeneous compute resources are the way to move forward. Another important trend, especially with the introduction of non-blocking collective communication in the latest MPI standard, is overlapping communication with computation. It has become an important design goal for messaging libraries like MVAPICH2 and OpenMPI. In this paper, we present an important benchmark that allows the users of different MPI libraries to evaluate performance of GPU-Aware Non-Blocking Collectives. The main performance metrics are overlap and latency. We provide insights on designing a GPU-Aware benchmark and discuss the challenges associated with identifying and implementing performance parameters like overlap, latency, effect of `MPI_Test()` calls to progress communication, effect of independent GPU communication while the overlapped computation proceeds under the communication, and the effect of complexity, target, and scale of this overlapped computation. To illustrate the efficacy of the proposed benchmark, we provide a comparative performance evaluation of GPU-Aware Non-Blocking Collectives in MVAPICH2 and OpenMPI.

Keywords

GPU-Aware, Micro-Benchmarking, MVAPICH2, Non-Blocking Collectives, OMB

1. INTRODUCTION

Two important trends are happening in cluster computing; first, the steady growth of accelerator based heterogeneous compute clusters and second, the introduction of Non-Blocking Collective (NBC) operations in the Message Passing Interface (MPI) standard. Both

are considered emerging strategies to implement applications for next-generation exascale architectures.

There has been a steady growth in the number of accelerator based compute clusters. The major vendor to provide these accelerators for High Performance Computing (HPC) has been NVIDIA. The latest Top500 (June 2015) [14] list has 48 out of 500 systems that use NVIDIA GPUs to accelerate performance of HPC applications. We believe that both the number and the complexity of such heterogeneous accelerators will continue to increase. The MPI community has responded well to this trend. Open source MPI libraries like MVAPICH2 [15] and OpenMPI [25] as well as proprietary libraries like Platform MPI [10] and Cray Message Passing Toolkit (MPT) [5] have redesigned their implementations to allow transparent and unified access to Device or GPU buffers for the point-to-point, one-sided, and blocking collective MPI routines. MVAPICH2 and OpenMPI have extended this support for some of the recently introduced NBC routines as well. Programmers can seamlessly switch between Host (CPU) and Device (GPU) buffers in their NBC MPI calls. We note that the term GPU-Aware refers to the fact that the underlying messaging middleware, used by an application, provides a unified CPU/GPU interface so that programmers do not have to handle explicit data movement and management between CPU and GPU. We also note that we use the terms Host and Device interchangeably for CPU and GPU respectively.

Most HPC applications have been using blocking collective communication operations like `MPI_Allgather()`, `MPI_Bcast()`, and `MPI_Reduce()` for a long time. On the other hand, GPU based applications have largely been designed for a single compute node with one or two GPUs. However, most modern applications are being designed for scale. This can effectively be achieved by combining the power of MPI for communication across nodes and the teraflop compute power of GPUs for performance inside a node. In this paper, our goal is to provide useful and analytical insights that help in understanding design and performance goals for such hybrid applications. Our main focus is towards applications that want to leverage the potential of GPU-Aware NBC operations like `MPI_Iallgather()`, `MPI_Iallreduce()`, and `MPI_Ibcast()`, etc. To the best of our knowledge, there exists no standard micro-benchmark or any performance analysis tool that explicitly focuses on evaluation of GPU-Aware NBC MPI implementations.

It is pertinent to mention that our group's earlier effort, OMB-GPU [3], was focused on GPU-Aware benchmarks for point-to-point, one-sided, and blocking collective communication only. In this paper, we extend our widely used OSU Micro-Benchmarks (OMB) [17] suite to address performance evaluation of GPU-Aware

*This research is supported in part by National Science Foundation grants #OCI-1148371, #CCF-1213084, and #CNS-1419123.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

EuroMPI '15, September 21-23, 2015, Bordeaux, France

© 2015 ACM. ISBN 978-1-4503-3795-3/15/09...\$15.00

DOI: <http://dx.doi.org/10.1145/2802658.2802672>

NBC communication routines. We allow the user to provide multiple switches to select the type and scale of evaluation parameters. The details of these parameters are discussed at length in Section 4. We make the following key contributions.

1. Present the design and implementation of the proposed GPU-Aware Non-Blocking Collective Benchmarks.
2. Provide useful insights on designing an effective benchmark for GPU-Aware NBC operations by discussing performance metrics like overlap and latency, communication progress mechanisms in MPI libraries, and independent CPU-GPU communication.
3. Discuss usage and performance effects of different runtime parameters including support for dummy compute on CPU, dummy compute on GPU, and independent CPU/GPU communication.
4. Illustrate the efficacy of our benchmarks by providing a comprehensive performance comparison of NBC operations in MVAPICH2 and OpenMPI on a GPU cluster.

To the best of our knowledge, this is the first effort towards providing a GPU-Aware benchmark for evaluating latency and overlap of NBC operations in a standardized manner.

2. BACKGROUND

We discuss some relevant background necessary to understand the work. We also introduce some useful terminologies.

2.1 Motivation for Non-Blocking Collectives

As compute clusters become bigger and faster, we are witnessing an ever increased focus on sub-microsecond communication latency. Undisputedly, the entire focus has been to reduce the latency between communicating processes. However, an orthogonal approach to this is to add one more parameter to the mix and focus on improving both of them instead of just one. The new parameter is the **overlap** of computation with communication. This leads to newer designs and implementations of the MPI standard. NBC implementations have existed before they were formally made part of the MPI standard. LibNBC [7] was the first library to introduce the concept. With MPI-3, NBCs have become part of the specification. Examples of NBCs include `MPI_Igather()` and `MPI_Iscatter()`. These enable application programmers to initiate an NBC operation and proceed with other useful (independent) computation. The amount of computation that can be performed while the communication progresses in the background is usually termed as the overlap. There are mechanisms that progress the communication in the background [9, 12, 18, 21]. The non-blocking semantics of NBCs lead to focus not just on minimizing the latency but also maximizing the overlap of computation and communication. Some researchers [7, 24] argue that a slight increase in latency is acceptable provided we get a greater degree of overlap. Figure 1 shows possible scenarios for GPU-Aware NBC operations.

The authors of LibNBC provide a benchmark called NBCBench [8] to evaluate the performance of LibNBC. OpenMPI and MVAPICH2 also provide implementations for NBC operations. Despite the fact that NBC implementations exist, there has been little work focused on evaluating their GPU-Aware implementations in a standardized manner. Moreover, as MPI libraries provide GPU-Aware implementations, evaluating the performance and overhead of NBC operations becomes a bigger challenge. NBCBench measures the overlap in LibNBC in an effective manner but the approach differs from prevalent micro-benchmarking suites like Intel MPI Benchmarks (IMB) [11] and OSU Micro-Benchmarks (OMB) [17]. We note that NBCBench is limited to Host based NBC operations only.

2.2 Programming for GPU based Clusters

Heterogeneous clusters with GPUs provide excellent throughput and thus are becoming increasingly popular. The nodes of the cluster are connected with high-bandwidth, low-latency network interfaces like InfiniBand (IB). Newer IB and GPU hardware also have GPUDirect RDMA (GDR) support. The NBC benchmarks proposed in this paper can readily take advantage of MVAPICH2-GDR and similar MPI implementations. The CUDA programming model still requires the data to be read/write back and forth between host and device but most implementations do this internally. The GPU-Aware NBC presented in this paper assume that the underlying implementation has the support for transparent CUDA-Aware MPI operations.

In a heterogeneous cluster, GPUs are connected to the CPU, inside a node, as a peripheral device via the PCI express bus. The nodes are connected by a high performance interconnect like InfiniBand or High Speed Ethernet. Using the CUDA programming model, GPUs can read/write memory attached to the local node. But it is necessary to move the data from the GPU (Device) memory to the Host’s memory before sending the data to the remote node. MPI is used to communicate the data across nodes. Multiple programming models for the data movement and management have increased the programming complexity to a great extent. In MVAPICH2, GPU-GPU MPI communication uses the standard MPI interfaces to unify the device memory and the host memory. It hides the GPU-CPU data movement on the sender’s side and optimizes GPU-CPU data movement and management inside an MPI library.

3. RELATED WORK

There has been a significant interest about micro-benchmarking in the HPC community. As a result, many benchmarks have been proposed to evaluate performance for general purpose GPU based applications. Parboil benchmark [23] and Rodinia benchmark [4] are application level benchmarks. Rodinia is designed to benchmark a diverse range of applications on heterogeneous computing infrastructures with OpenMP, OpenCL, and CUDA implementations. Parboil also provides applications (benchmarks) that focus on a variety of application areas.

OMB-GPU was developed to evaluate GPU-Aware MPI libraries. The focus here was on the common communication models in GPU-Aware MPI applications that use Pt-to-Pt, One-Sided, and Blocking collective operations and utilize GPU accelerators. The Scalable Heterogeneous Computing (SHOC) [6] benchmark suite has been proposed to test the performance and stability of the heterogeneous clusters. SHOC-MIC [1] has also been proposed that ports the SHOC suite to the Intel MIC architecture. Currently, SHOC-MIC is in alpha and does not support MPI as yet.

In most of the existing MPI benchmark suites – NBCBench, Intel’s IMB, Sandia MPI Benchmarks (SMB) [22], Communication Offload MPI-based Benchmark (COMB) [13], NAS Parallel Benchmarks [2], and SPECMPI [16] – both the source and the destination buffers reside in the host’s memory. Table 1 summarizes the features in different benchmark suites discussed in the related work. To compare NBC-specific features, we compare feature of IMB, NBCBench, and our Proposed benchmarks in Table 2. Programmers can modify these benchmarks to evaluate them on GPU clusters by adding explicit CPU-GPU data movement and management code. However, it is not a very convenient and optimized solution. Prior research [26] highlights that data movement in and out of GPGPUs remains the biggest hurdle to overall performance and programmer productivity. The purpose of the benchmarks we have proposed in this paper is to provide the programmers a standard suite that evaluates non-blocking collective operations in different

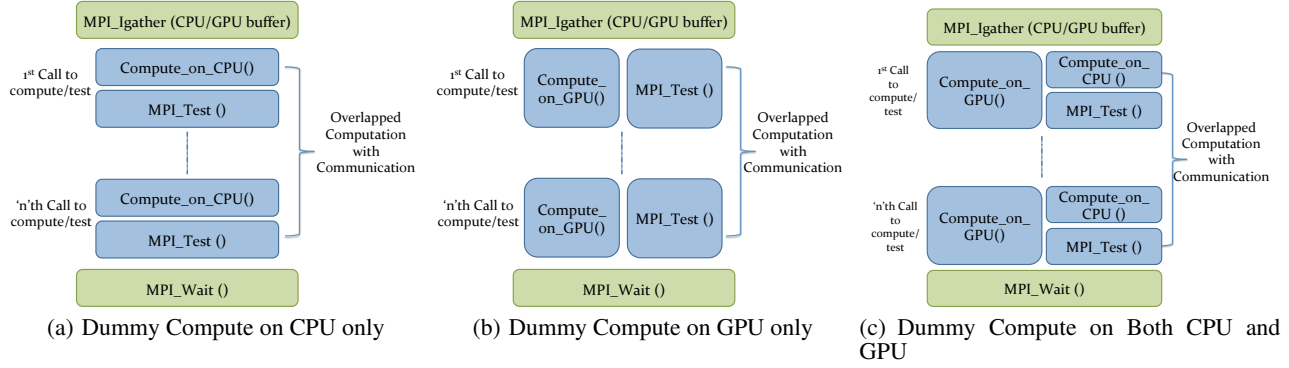


Figure 1: Common Scenarios for using GPU-Aware Non-Blocking Collectives

Benchmarks/Features	Pt-to-Pt, One-Sided		Blocking Collectives		Non-Blocking Collectives	
	Host-based	GPU-Aware	Host-based	GPU-Aware	Host-based	GPU-Aware
IMB [11]	✓	✗	✓	✗	✓	✗
COMB [13]	✓	✗	✓	✗	✗	✗
SMB [22]	✓	✗	✓	✗	✗	✗
NBCBench [8]	✗	✗	✗	✗	✓	✗
OMB [17]	✓	✓	✓	✓	✗	✗
OMB (w/ Proposed GPU-NBC)	✓	✓	✓	✓	✓	✓

Table 1: Features Comparison Between Existing and Proposed Benchmarks

MPI libraries.

At the same time, since there are many factors to affect the data communication performance, such as the pinned or pageable host memory, the exclusive or shared usage for one GPU, the I/O Hub (shared or not shared) for GPUs and NICs, and so on, it is hard to write the test programs and do a fair comparison on different MPI libraries in an adhoc manner. In addition, it is necessary to identify these important architecture related parameters and understand how the benchmarks respond to changes of these parameters. The benchmarks proposed in this paper alleviate these issues and allow the users to simply evaluate different MPI implementations using different runtime parameters.

4. DESIGNING GPU-AWARE NBC BENCHMARKS

In this section, we elaborate on the design choices made for implementing non-blocking collective benchmarks. First, we discuss the significance of evaluating overlap of computation and computation. Second, we describe the dummy compute function we have added to mimic real world applications that perform useful computation while the communication proceeds in the background. In addition, we also discuss the dummy compute function executed as a CUDA kernel on the GPU. Third, we describe the use of `MPI_Test()` calls interleaved with dummy computation to imitate the behavior of applications that manually progress communication using `MPI_Test()` or `MPI_Probe()` calls while performing computation. Fourth, we describe a dummy copy function that has been designed to perform independent GPU-CPU and CPU-GPU communication using `cudaMemcpyAsync` calls. Fifth and last, we address cases where a GPU application accesses a large amount of data from the GPU's main memory very frequently. At the end of this section, we provide a summary of the new runtime (command line) options we have added.

4.1 Overlapping Communication and Computation

The primary goal for non-blocking collectives in the MPI standard is to provide an alternate interface for collective communi-

cation in applications that require overlapped communication and computation. Precisely, we want to perform useful computation while the communication can proceed in the background unlike blocking point-to-point and collective operations. Several MPI implementations [15, 25] and applications [19, 20] have been developed that highlight the importance of an increased overlap.

```

for(i = 0; i < iterations; i++) {
    t_start = MPI_Wtime();

    MPI_Igather(sendbuf, size, MPI_CHAR,
               recvbuf, size, MPI_CHAR,
               0, MPI_COMM_WORLD, &request);
    MPI_Wait(&request, &status);

    t_stop = MPI_Wtime();

    timer += t_stop - t_start;
}

MPI_Barrier(MPI_COMM_WORLD);

latency = timer / iterations;

```

Listing 1: Pure Communication Latency

```

/* for loop with dummy_compute */
for(i=0; i < iterations; i++) {
    t_start = MPI_Wtime();

    init_time = MPI_Wtime();
    MPI_Igather(sendbuf, size, MPI_CHAR,
               recvbuf, size, MPI_CHAR,
               0, MPI_COMM_WORLD, &request);
    init_time = MPI_Wtime() - init_time;

    tcomp = MPI_Wtime();
    test_time = dummy_compute(comm_latency, 0,
                              &request);
    tcomp = MPI_Wtime() - tcomp;
}

```

Benchmarks/Eval. Metrics	Overlap	Latency	MPI_Test () calls	MPI_Wait ()	Dummy Compute on CPU	Dummy Compute on GPU	Dummy Copy
IMB [11]	✓	✓	✗	✗	✓	✗	✗
NBCBench [8]	✓	✓	✓	✓	✓	✗	✗
OMB (w/ Proposed GPU-NBC)	✓	✓	✓	✓	✓	✓	✓

Table 2: NBC-specific Benchmark Features Comparison

```

wait_time = MPI_Wtime();
MPI_Wait(&request, &status);
wait_time = MPI_Wtime() - wait_time;

t_stop = MPI_Wtime();

accumulate_timers();
}

```

Listing 2: The Overlapped Benchmark Loop

```

dummy_compute(double latency, int init,
              MPI_Request* request)
{
    if (init) {
        init_arrays();
    } else {
        test_time = do_compute_and_test(latency,
                                         request);
    }
}

```

Listing 3: The Dummy Compute Function

```

double do_compute_and_test(double latency,
                          MPI_Request* request)
{
    if (options.num_tests)
        target_for_compute =
            latency/options.num_tests;
}

if (options.target == gpu) {
    if (options.num_tests) {
        do_compute_gpu(target_for_compute);

        for (options.num_tests)
            MPI_Test();
    } else {
        do_compute_gpu(latency);
    }
}

if (options.target == both) {
    if (options.num_tests) {
        do_compute_gpu(target_for_compute);

        do_compute_cpu(target_for_compute);

        for (options.num_tests)
            MPI_Test();
    } else {
        do_compute_gpu(latency);
        do_compute_cpu(latency);
    }
}

if (options.target == cpu) {
    if (options.num_tests) {
        do_compute_cpu(target_for_compute);

        for (options.num_tests)
            MPI_Test();
    } else {
        do_compute_cpu(latency);
    }
}

```

```

}
}

```

Listing 4: The Compute and Test Function

In order to evaluate the overlap, we first use a loop to calculate pure communication latency for an NBC (e.g. `MPI_Ibcast()`) using a call to `MPI_Ibcast()` immediately followed by an `MPI_Wait()` call. This latency is pre-computed and is not considered a part of the actual evaluation. Listing 1 shows the code for this calculation. We later use this latency to call a dummy compute function as shown in Listing 2. We note that `latency` in Listing 1 is the same as `comm_latency` in Listing 2. The pseudo-code for the inner loop in our benchmark basically does three things. First, it calls an NBC e.g. `MPI_Igather()`. Second, it calls a dummy compute routine (the details of this routine are in Section 4.2). Third, it calls `MPI_Wait()`. The inner loop is inside a loop for different message sizes that can be specified by a command line parameter (`-m SIZE`).

4.2 Dummy Compute Kernel for Host and Device

In order to show the usefulness of NBCs, we use a dummy computation function in our code. We note that this function can be a simple call to `sleep()` but we use a cache unfriendly code to mimic the worst-case scenario of a real world application that performs useful computation in an overlapped manner. We use dynamically allocated 2-dimensional arrays. We access one of them in row-major order while the other one in column-major order to counter possible compiler optimization. This compute function also has the facility to interleave `MPI_Test()` calls in the code at regular intervals. The number of calls can be set using a command line parameter (`-t CALLS`). The motivation to use `MPI_Test()` calls in the code is described in Section 4.3. Listing 4 contains the main code. It basically handles six different combinations based on the common scenarios shown in Figure 1. We need to perform three different operations; dummy compute on CPU only, dummy compute on GPU only, and dummy compute on both CPU and GPU. In addition, these change if the user has specified to use `MPI_Test()` or not. This is checked in the code using the `options.num_tests` field. We implement the first using a compute on Host and `MPI_Test()` calls at regular intervals. The calculation is shown in Listing 4. The computation target time that is pre-computed is divided by the number of `MPI_Test()` calls. We simply loop for the number of tests. The second configuration (GPU only) is implemented using an asynchronous kernel launch on the GPU and performing the specified number of tests on the Host only. Third configuration (Both CPU and GPU) is implemented by an asynchronous launch of a kernel, followed by a loop similar to the Host only configuration.

4.3 Explicit Communication Progress

MVAPICH2 uses an internal progress engine that can be queried / progressed using `MPI_Test()` or `MPI_Probe()` calls. Other MPI libraries also have similar mechanisms. It is important for an application to determine an appropriate number of `MPI_Test()` calls that are required to progress the communication while it performs some independent useful computation. Our benchmark provides the user with the facility to experiment and determine the correct number of `MPI_Test()` calls. We have used 0 (means no manual progress), 1000, and 10000 as the number of these calls in

our results.

4.4 Dummy Copy Function

The motivation for dummy copy function is to study the effect of independent GPU to CPU and CPU to GPU communication while a dummy compute kernel is being executed on the GPU and an NBC collective in progress. This can be enabled using a flag (-p) to the benchmark. We are providing this as an experimental feature. We note that we only provide a small study for this feature in our paper. An in-depth investigation will be conducted in the future.

4.5 Summary of New Runtime Options

We have added several runtime options for the proposed benchmarks. First, we utilized the -d CUDA flag that was available in OMB-GPU. Based on this, we allocate the source and destination buffers on the device. Second, we have introduced -r TARGET. The user can specify the dummy computation target using -r CPU, GPU, or BOTH. In our code, we use `dummy_compute()` and `call_kernel()` functions to achieve the desired purpose. In the `call_kernel()` function, we call a CUDA kernel using a separate stream in an asynchronous fashion. The `dummy_compute()` function has been explained earlier in the Section 4.2. These two functions can be used as a reasonable imitation of a real application's workload. Third, we introduced -s SIZE and -x SIZE that specify the size of arrays used by the `call_kernel()` and `dummy_copy()` functions. Fourth, we have added -t NUMCALLS for taking user input on the number of `MPI_Test()` calls to be interleaved in the dummy computation function.

5. PERFORMANCE COMPARISON

5.1 Experimental Setup

Wilkes cluster, deployed in Nov 2013 at Cambridge, U.K., has been used for the performance evaluation. The cluster is partitioned with different configurations. For our purpose we use the Tesla partition which has 128 nodes. Each node has a 6-core dual-socket Intel IvyBridge processor. The nodes are equipped with 2 Tesla K20 NVIDIA GPUs and 2 FDR IB HCAs.

5.2 Comparative Performance Evaluation of MPI libraries

We believe that it is important to demonstrate the potential use-cases and/or benefits of the proposed benchmarks. We provide a comprehensive performance comparison for two widely used MPI libraries. We used CUDA 6.5 with MVAPICH2.1a and OpenMPI 1.8.4. Appropriate GDR flags were used for both the libraries. The broad categories of the evaluation are overlap and latency for these two MPI libraries. Further, we study the effect of different parameters like number of `MPI_Test()` calls and target of dummy computation on both latency and overlap. For the sake of relevance, we picked the most widely used following collectives for our evaluation.

`MPI_Iallgather()`, `MPI_Ialltoall()`, `MPI_Ibcast()`, and `MPI_Iallreduce()` were evaluated on Wilkes. We note that we successfully evaluated `MPI_Iallreduce()` for MVAPICH2 but we were not able to execute the tests for OpenMPI so we have omitted the graphs for it. We have restricted our study to 8 nodes and 16 processes (2 processes per node). We ran into some errors when we executed GPU-Aware NBC operations on 32 and 64 nodes of the Wilkes cluster. We note that all the results shown in the paper are for `MPI_Ibcast()` only. For other collectives, we have the numbers and the graphs but they have been omitted due to space restrictions.

5.3 Overhead Comparison for Blocking and Non-Blocking Operation

We first compare the overhead posed by a non-blocking collective. We demonstrate the overhead on latency by comparing total (overall) latency of the benchmark with pure communication latency. Figure 2 shows results for both OpenMPI and MVAPICH2. We observed that the overhead for small message sizes is around 20 percent for both OpenMPI and MVAPICH2. An interesting insight here is that MVAPICH2 experiences almost no overhead for messages in the range of 32 KB to 512 KB. OpenMPI experiences overhead in that range but no overhead for message sizes between 256 KB and 1 MB. This can be seen from Figure 2(b).

5.4 Effect of Parameters on Overlap and Latency

The interesting part of the benchmarks is the effect of different parameters that they have on performance of the NBC operations.

5.4.1 Effect of Dummy Copy

The dummy copy (independent CPU-GPU communication) function increases the latency and the overlap. Figure 3 shows this effect for both OpenMPI and MVAPICH2. We attribute this overhead to the impact of the copy on the memory usage of the GPU which affects the kernel execution time. We note that the copy is an asynchronous copy and uses a different stream than the compute kernel stream. In fact, we use a separate stream for copying from CPU to GPU and from GPU to CPU. We show the effect of dummy communication (copy) for small message size latency. We did not observe a significant effect for other configurations.

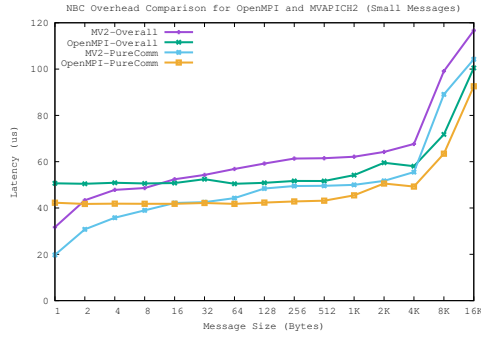
5.4.2 Number of `MPI_Test()` Calls

Various implementations of non-blocking operations periodically call `MPI_Test()` to progress communication in the background. Calling `MPI_Test()` increases the overlap but may increase latency for small message sizes. It affects the CPU time because of the dummy (overlapped computation) and tests happening on the CPU simultaneously. It may also affect the cache behavior during the application's computation phase. Thus the usage of `MPI_Test()` is a trade off between overlap and latency. For convenience, the number of test calls have been added as a selectable runtime parameter. Figures 4 and 5 show the effect of varying the number of test calls from 1,000 (1K) to 10,000 (10K). We observe that the effect is significant for both small and medium message sizes.

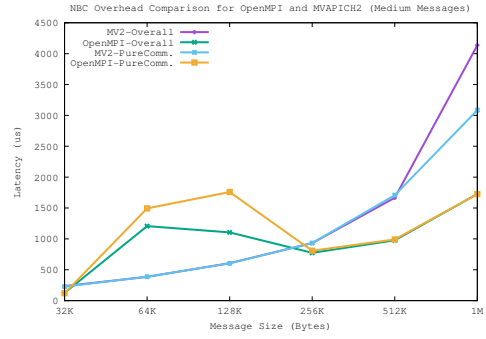
We observed that latency of small messages increased significantly as we increased the number of test calls from 1K to 10K. Both, OpenMPI and MVAPICH2, experience a similar overhead of test calls. This is shown in Figure 4(a). This is an expected outcome as the benefit of progressing the communication using test calls is shadowed because of the very small latencies for smaller messages. Hence it becomes an overhead instead of benefit. Overlap for small messages was largely unaffected in both OpenMPI and MVAPICH2. This can be seen from Figure 4(b) that demonstrates benefits in some ranges and overhead in certain ranges.

For medium message sizes, MVAPICH2 demonstrated near perfect overlap with 10K test calls. OpenMPI also achieved around 90 percent overlap for certain message sizes in the medium range but a certain drop was observed for 1 MB messages. This trend can be seen from Figure 5(b). Interestingly, the latency for larger messages improved with an increased number of test calls. Figure 5(a) shows that MVAPICH2 achieved much lower latency for messages in the medium range as we increased the number of tests to 10K. For OpenMPI we did not observe any noticeable change in the latency in the medium message range.

5.4.3 Overlapped Computation Target

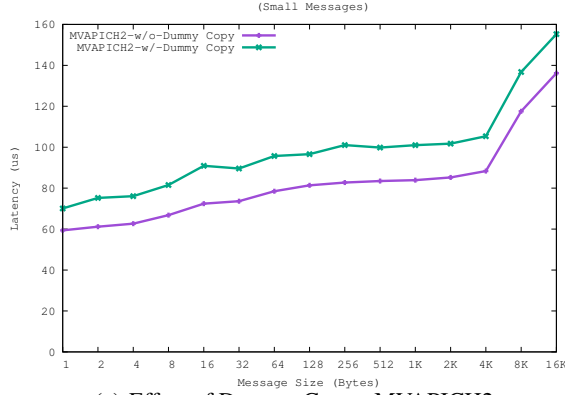


(a) NBC Overhead for Small Message Sizes

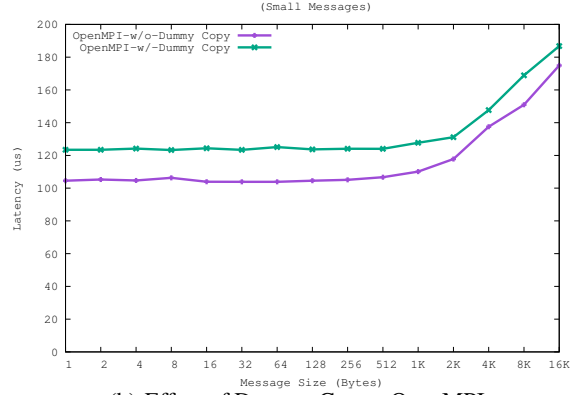


(b) NBC Overhead for Medium Message Sizes

Figure 2: NBC Overhead for Non-Blocking MPI_Ibcast () with Dummy Compute on CPU

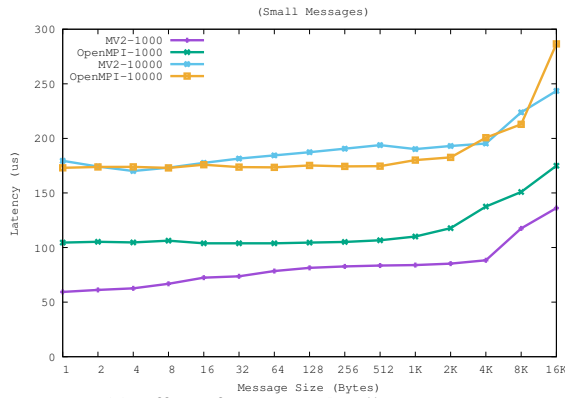


(a) Effect of Dummy Copy - MVAPICH2

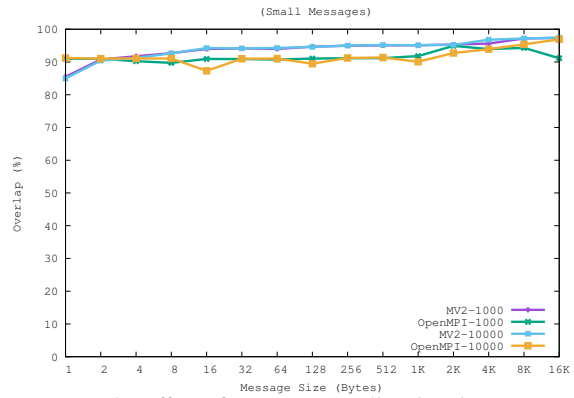


(b) Effect of Dummy Copy - OpenMPI

Figure 3: Effect of Dummy Copy on Latency of Non-Blocking MPI_Ibcast () with Dummy Compute on GPU

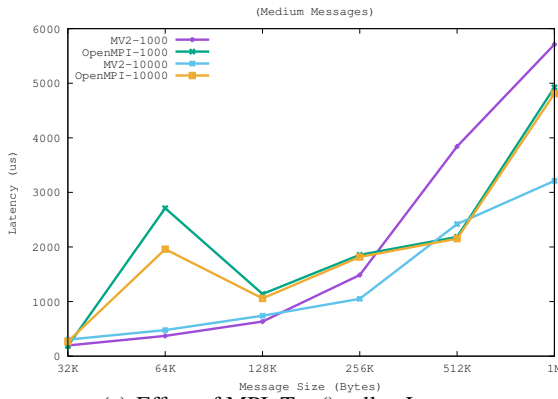


(a) Effect of MPI_Test() calls - Latency

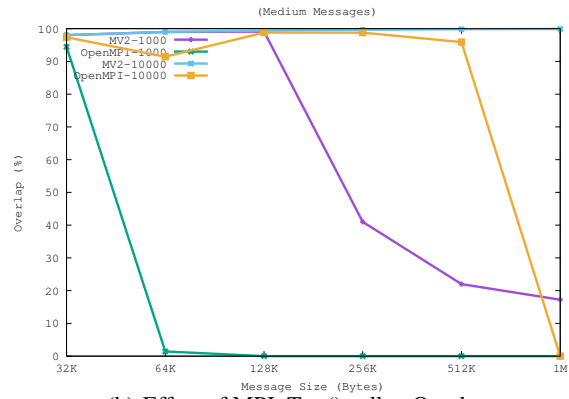


(b) Effect of MPI_Test() calls - Overlap

Figure 4: Effect of MPI_Test() calls on Latency and Overlap of Non-Blocking MPI_Ibcast ()



(a) Effect of MPI_Test() calls - Latency



(b) Effect of MPI_Test() calls - Overlap

Figure 5: Effect of MPI_Test() calls on Latency and Overlap of Non-Blocking MPI_Ibcast ()

The dummy compute function (as mentioned earlier) is at the heart of this benchmark. With six different combinations, we provide the latency numbers for dummy compute on CPU, dummy compute on GPU, and dummy compute on both CPU and GPU in Figures 6, 8, and 10. The overlap for these three configurations is illustrated in Figures 7, 9, and 11. In addition, the suffixes (-1000) and (-0) refer to the number of test calls used. A 0 (zero) here means that no test call was made. The code-path taken for this is different than it is taken with test calls greater than zero.

We observed that the latencies were mostly predictable and stable. The overlap numbers do not follow a predictable pattern. Overlap increased as we increased the number of test calls from 0 to 1K. This was observed for both OpenMPI and MVAPICH2 in small as well as medium size ranges. The effect was significant for some configurations but in general the overlap increased. Latency decreased under most configurations as we increased the number of tests to 1,000. We noticed slightly higher latencies for medium message sizes when we moved from compute on CPU to compute on GPU only configuration. This can be compared using Figures 6(b) and 8(b). The most interesting result, and perhaps worth further investigation, was overlap for small message sizes. We observed near-zero overlap for small message sizes in compute on GPU only, and compute on both CPU and GPU configurations when no test calls were made. With number of tests increased to 1,000, we saw an excellent 80-90 percent overlap for both OpenMPI and MVAPICH2. This can be seen from Figures 8(a) and 10(a). Overall, best latency and overlap was achieved across all configurations was for MVAPICH2 with 1,000 test calls.

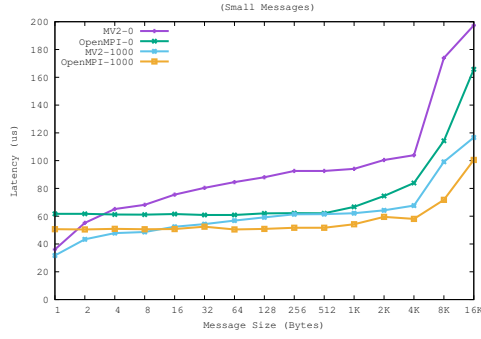
6. CONCLUSION AND FUTURE WORK

In this paper, we present new benchmarks to evaluate non-blocking collectives (NBC) on GPU clusters. We first argue that we need GPU-Aware benchmarks for NBCs because more and more clusters are adding accelerators to improve performance of high performance applications and libraries. Different NBC operations like `MPI_Allgather()`, `MPI_Ibcast()` etc. became part of the MPI-3 standard recently. Libraries like MVAPICH2 and OpenMPI provide efficient GPU-Aware implementations for most of the MPI routines including non-blocking collectives. Proprietary MPI libraries also support GPU-Aware NBC and can be also be evaluated using our benchmark. This is the primary motivation for a standard benchmark that provides a means of fair evaluation across different MPI libraries. We also highlight new evaluation metrics and methods like pure communication time, overlap of computation with communication, time for `MPI_Wait()` and `MPI_Test()` calls, and time for an adaptive dummy compute function that mimics real

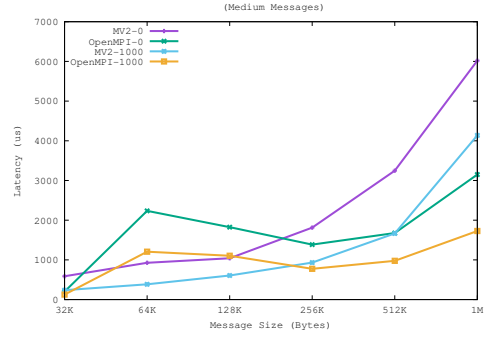
world application scenarios discussed in the introduction. For completeness, we provide a detailed performance comparison of MVA-PICH2 and OpenMPI using the new NBC benchmarks and the new parameters. We plan to add these benchmarks to the widely used OMB suite and make them publicly available as part of a future release.

References

- [1] SHOC for Intel MIC. <https://github.com/vetter/shoc-mic>, 2015.
- [2] D. H. Bailey, E. Barszcz, J. T. Barton, D. S. Browning, R. L. Carter, L. Dagum, R. A. Fatoohi, P. O. Frederickson, T. A. Lasinski, R. S. Schreiber, et al. The NAS Parallel Benchmarks. *IJHPCA*, 5(3):63–73, 1991.
- [3] D. Bureddy, H. Wang, A. Venkatesh, S. Potluri, and D. Panda. OMB-GPU: A Micro-Benchmark Suite for Evaluating MPI Libraries on GPU Clusters. In J. Träff, S. Benkner, and J. Dongarra, editors, *Recent Advances in the Message Passing Interface*, volume 7490 of *Lecture Notes in Computer Science*, pages 110–120. Springer Berlin Heidelberg, 2012. ISBN 978-3-642-33517-4. URL http://dx.doi.org/10.1007/978-3-642-33518-1_16.
- [4] S. Che, J. Sheaffer, M. Boyer, L. Szafaryn, L. Wang, and K. Skadron. A characterization of the Rodinia benchmark suite with comparison to contemporary CMP workloads. In *Workload Characterization (IISWC), 2010 IEEE International Symposium on*, pages 1–11, Dec 2010.
- [5] Cray. Cray Message Passing Toolkit. <http://docs.cray.com/books/S-9407-1305/S-9407-1305.pdf>, 2015.
- [6] A. Danalis, G. Marin, C. McCurdy, J. S. Meredith, P. C. Roth, K. Spafford, V. Tipparaju, and J. S. Vetter. The Scalable Heterogeneous Computing (SHOC) Benchmark Suite. In *Proceedings of the 3rd Workshop on General-Purpose Computation on Graphics Processing Units, GPGPU '10*, pages 63–74, New York, NY, USA, 2010. ACM. ISBN 978-1-60558-935-0. URL <http://doi.acm.org/10.1145/1735688.1735702>.
- [7] T. Hoefler, A. Lumsdaine, and W. Rehm. Implementation and Performance Analysis of Non-blocking Collective Operations for MPI. In *Supercomputing, 2007. SC'07. Proceedings of the 2007 ACM/IEEE Conference on*, pages 1–10. IEEE, 2007.

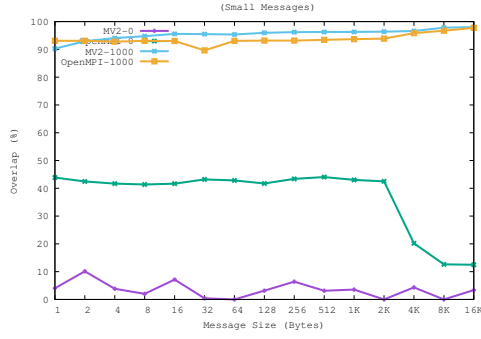


(a) Latency for Small Message Sizes

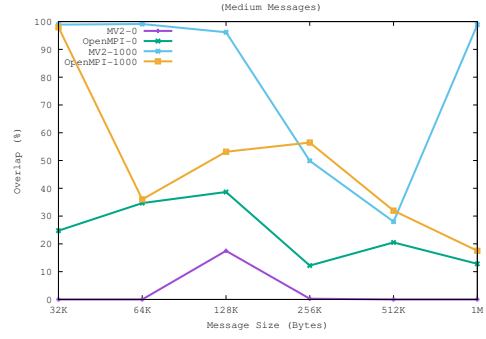


(b) Latency for Medium Message Sizes

Figure 6: Non-Blocking MPI_Ibcast () Latency with Dummy Compute on CPU only

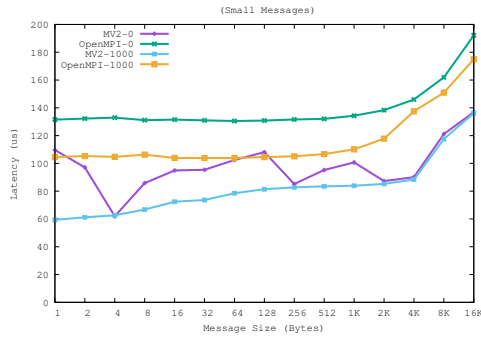


(a) Overlap for Small Message Sizes

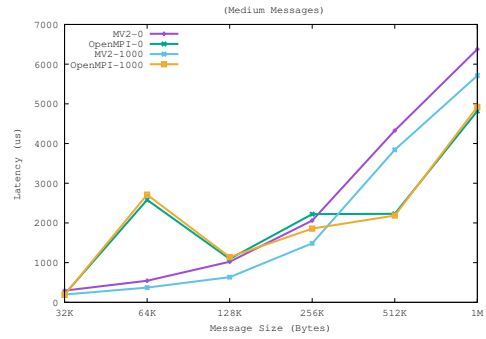


(b) Overlap for Medium Message Sizes

Figure 7: Non-Blocking MPI_Ibcast () Overlap with Dummy Compute on CPU only

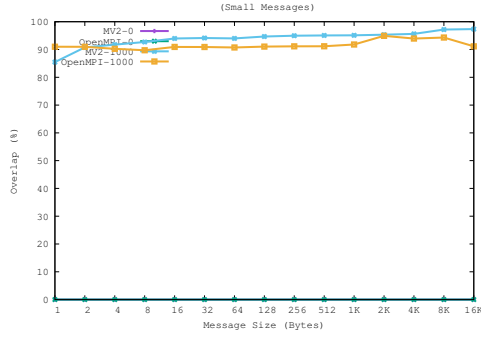


(a) Latency for Small Message Sizes

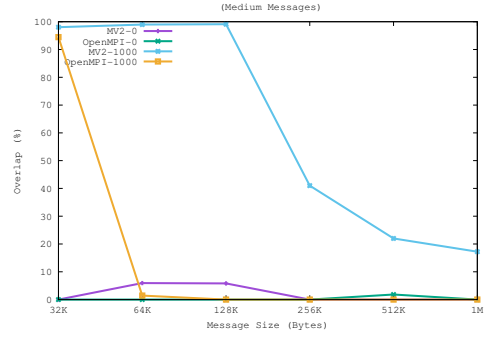


(b) Latency for Medium Message Sizes

Figure 8: Non-Blocking MPI_Ibcast () Latency with Dummy Compute on GPU only

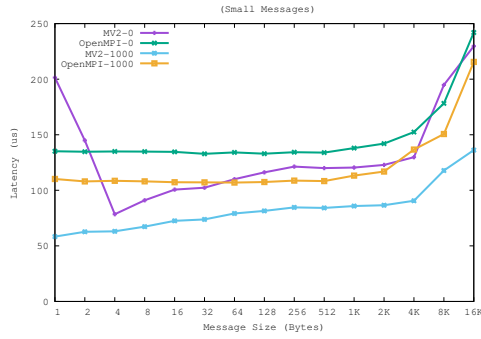


(a) Overlap for Small Message Sizes

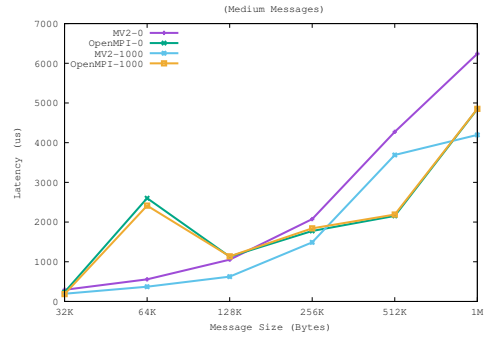


(b) Overlap for Medium Message Sizes

Figure 9: Non-Blocking MPI_Ibcast () Overlap with Dummy Compute on GPU only

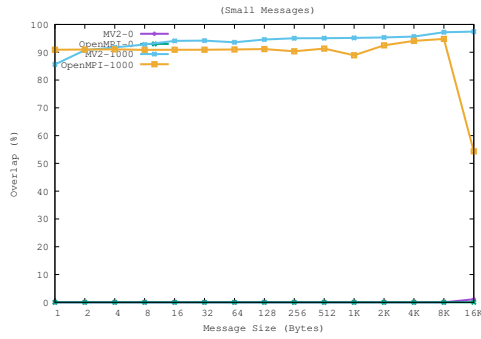


(a) Latency for Small Message Sizes

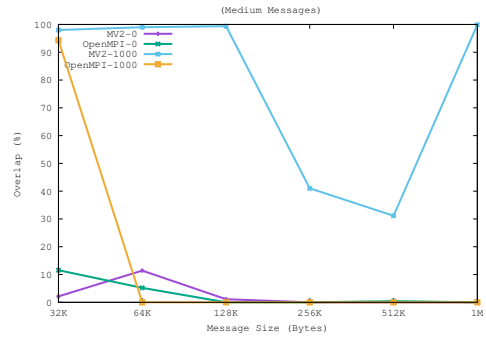


(b) Latency for Medium Message Sizes

Figure 10: Non-Blocking MPI_Ibcast () Latency with Dummy Compute on Both CPU and GPU



(a) Overlap for Small Message Sizes



(b) Overlap for Medium Message Sizes

Figure 11: Non-Blocking MPI_Ibcast () Overlap with Dummy Compute on Both CPU and GPU

- [8] T. Hoeﬂer, T. Schneider, and A. Lumsdaine. Accurately Measuring Collective Operations at Massive Scale. In *Proceedings of the 22nd IEEE International Parallel & Distributed Processing Symposium, PMEO'08 Workshop*, Apr. 2008. ISBN 978-1-4244-1694-3.
- [9] T. Hoeﬂer, C. Siebert, and A. Lumsdaine. Group Operation Assembly Language - A Flexible Way to Express Collective Communication. In *ICPP-2009 - The 38th International Conference on Parallel Processing*. IEEE, Sep. 2009. ISBN 978-0-7695-3802-0.
- [10] IBM. Platform MPI. <http://www-03.ibm.com/systems/platformcomputing/products/mpi/index.html>, 2015.
- [11] IMB. Intel MPI Benchmarks (IMB). <https://software.intel.com/en-us/articles/intel-mpi-benchmarks>.
- [12] K. C. Kandalla, H. Subramoni, K. Tomko, D. Pekurovsky, and D. K. Panda. A Novel Functional Partitioning Approach to Design High-Performance MPI-3 Non-blocking Alltoallv Collective on Multi-core Systems. In *42nd International Conference on Parallel Processing, ICPP 2013, Lyon, France, October 1-4, 2013*, pages 611–620, 2013. . URL <http://dx.doi.org/10.1109/ICPP.2013.75>.
- [13] W. Lawry, C. Wilson, A. B. Maccabe, and R. Brightwell. COMB: A Portable Benchmark Suite for Assessing MPI Overlap. In *IEEE Cluster*, pages 23–26, 2002.
- [14] H. Meuer, E. Strohmaier, J. Dongarra, and H. Simon. TOP 500 Supercomputer Sites. <http://www.top500.org>.
- [15] MVAPICH2: MPI over InfiniBand, 10GigE/iWARP and RoCE. <https://mvapich.cse.ohio-state.edu/>.
- [16] M. S. Müller, M. van Waveren, R. Lieberman, B. Whitney, H. Saito, K. Kumaran, J. Baron, W. C. Brantley, C. Parrott, T. Elken, H. Feng, and C. Ponder. SPEC MPI2007 : An Application Benchmark Suite for Parallel Systems using MPI. *Concurrency and Computation: Practice and Experience*, 22(2):191–205, 2010. ISSN 1532-0634. . URL <http://dx.doi.org/10.1002/cpe.1535>.
- [17] Network Based Computing Laboratory. OSU Micro-Benchmarks. <http://mvapich.cse.ohio-state.edu/benchmarks/>, 2015.
- [18] A. Nomura and Y. Ishikawa. Design of Kernel-level Asynchronous Collective Communication. In *Proceedings of the 17th European MPI Users' Group Meeting Conference on Recent Advances in the Message Passing Interface, EuroMPI'10*, pages 92–101, Berlin, Heidelberg, 2010. Springer-Verlag. ISBN 3-642-15645-2, 978-3-642-15645-8. URL <http://dl.acm.org/citation.cfm?id=1894122.1894135>.
- [19] K. Olsen. *Simulation of three dimensional wave propagation in the Salt Lake Basin*. PhD thesis, The University of Utah, Salt Lake City, Utah, 1994.
- [20] D. Pekurovsky. P3DFFT: A Framework for Parallel Computations of Fourier Transforms in Three Dimensions. *SIAM Journal on Scientific Computing*, 34(4):C192–C209, 2012. . URL <http://dx.doi.org/10.1137/11082748X>.
- [21] T. Schneider, S. Eckelmann, T. Hoeﬂer, and W. Rehm. Kernel-Based Offload of Collective Operations - Implementation, Evaluation and Lessons Learned. In *Proceedings of the 17th international conference on Parallel processing - Volume Part II*, pages 264–275. Springer-Verlag, Aug. 2011. ISBN 978-3-642-23396-8.
- [22] SMB. Sandia MPI Micro-Benchmark Suite (SMB). <http://www.cs.sandia.gov/smb/index.html>.
- [23] J. A. Stratton, C. Rodrigues, I.-J. Sung, N. Obeid, L.-W. Chang, N. Anssari, G. D. Liu, and W. mei W. Hwu. Parboil: A Revised Benchmark Suite for Scientific and Commercial Throughput Computing. Technical report, University of Illinois at Urbana-Champaign, March 2012.
- [24] H. Subramoni, A. A. Awan, K. Hamidouche, D. Pekurovsky, A. Venkatesh, S. Chakraborty, K. Tomko, and D. K. Panda. Designing Non-blocking Personalized Collectives with Near Perfect Overlap for RDMA-Enabled Clusters. In J. M. Kunkel and T. Ludwig, editors, *High Performance Computing*, volume 9137 of *Lecture Notes in Computer Science*, pages 434–453. Springer International Publishing, 2015. ISBN 978-3-319-20118-4. . URL http://dx.doi.org/10.1007/978-3-319-20119-1_31.
- [25] The Open MPI Development Team. Open MPI : Open Source High Performance Computing. <http://www.open-mpi.org>.
- [26] H. Wang, S. Potluri, M. Luo, A. K. Singh, X. Ouyang, S. Sur, and D. K. Panda. Optimized Non-contiguous MPI Datatype Communication for GPU Clusters: Design, Implementation and Evaluation with MVAPICH2. In *Proceedings of the 2011 IEEE International Conference on Cluster Computing, CLUSTER '11*, pages 308–316, Washington, DC, USA, 2011. IEEE Computer Society. ISBN 978-0-7695-4516-5. . URL <http://dx.doi.org/10.1109/CLUSTER.2011.42>.